



Universidade Federal Fluminense
Disciplina: Análise e Projeto de Algoritmos
Professor: Luís Felipe

Revisão AP2 - com algumas ideias das respostas

1. Dado um polígono convexo com n vértices, queremos determinar o número mínimo de operações necessárias para dividi-lo completamente em triângulos, utilizando as seguintes regras:
 - A cada passo, escolhemos uma diagonal do polígono e a traçamos, dividindo o polígono em dois subpolígonos.
 - A separação é completamente balanceada, ou seja, cada um dos dois subpolígonos resultantes possui exatamente $\frac{n}{2}$ vértices.
 - O processo continua recursivamente nos dois subpolígonos até que todos sejam triângulos.

Obtenha $T(n)$, que representa o número de operações necessárias para particionar um polígono de n lados em triângulos. Além disso, obtenha a complexidade dessa abordagem recursiva.

Resposta: Recorrência geral $T(n) = 2T(n/2) + O(n)$

2. Desenvolva o algoritmo pelo método de divisão e conquista para obter o k -ésimo menor elemento de uma lista de inteiros, sem utilizar nenhum método de ordenação para sua resolução. Determine também suas complexidades de melhor caso e de pior caso.

Resposta: Conteúdo visto em aula.

3. Desenvolva um algoritmo que obtém o maior número possível de atividades que podem ser realizadas em um auditório. Além disso, determine a complexidade desse algoritmo. Cada atividade i tem um horário de início s_i e um horário de término f_i , onde $0 \leq s_i < f_i$.

O auditório pode ser usado para apenas uma atividade por vez, ou seja, duas atividades não podem se sobrepor no tempo. Seu objetivo é selecionar o maior número de atividades que possam ser realizadas sem conflitos.

Entrada:

- Um número inteiro n representando a quantidade de atividades.
- Uma lista de n pares (s_i, f_i) , onde s_i é o horário de início e f_i é o horário de término da atividade i .

Saída:

- O número máximo de atividades que podem ser selecionadas sem sobreposição.

Resposta: Conteúdo visto em aula.

4. Explique a importância da Programação Dinâmica na resolução de problemas computacionais, explicando como essa abordagem funciona. Além disso, apresente um exemplo de problema que pode ser resolvido utilizando Programação Dinâmica, detalhando a formulação da estratégia de solução.

5. Dado um conjunto de moedas de valores distintos $C = \{c_1, c_2, \dots, c_m\}$ e um valor-alvo N , determine o número mínimo de moedas necessárias para formar o valor N . Além disso, determine a complexidade desse algoritmo, que deve ser eficiente.

Exemplo:

- Entrada: Moedas disponíveis: $[1, 3, 4]$, Valor-alvo: 6
- Saída: Número mínimo de moedas: 2 (usando moedas $[3, 3]$ ou $[4, 1, 1]$)

Resposta: Seja $dp[v]$ o menor número de moedas necessário para totalizar exatamente o valor v . A relação de recorrência é definida como:

$$dp[0] = 0$$

$$dp[v] = \begin{cases} \min_{1 \leq i \leq n, c_i \leq v} (dp[v - c_i] + 1), & \text{se existir } c_i \leq v \\ \infty, & \text{caso contrário} \end{cases}$$

Se $dp[V] = \infty$, então não é possível formar o valor V com as moedas disponíveis, e a saída deve ser -1 .

6. Dado um conjunto de n itens, onde cada item i possui um peso $w_i \in \mathbb{N}$ e um valor $v_i \in \mathbb{N}$, e uma mochila com capacidade máxima $W \in \mathbb{N}$, determine o valor máximo total que pode ser obtido selecionando um subconjunto dos itens de forma que: a soma dos pesos dos itens escolhidos não exceda W ; cada item seja usado no máximo uma vez.

Entrada:

- Um vetor de inteiros $w = [w_1, w_2, \dots, w_n]$, representando os pesos dos itens;
- Um vetor de inteiros $v = [v_1, v_2, \dots, v_n]$, representando os valores dos itens;
- Um inteiro W , representando a capacidade máxima da mochila.

Saída:

- Um número inteiro representando o valor total máximo que pode ser obtido com os itens escolhidos.

Resposta:

Seja $dp[i][w]$ o valor máximo que pode ser obtido utilizando os primeiros i itens com capacidade máxima w . A relação de recorrência é dada por:

$$dp[0][w] = 0 \quad \forall w \in [0, W]$$

$$dp[i][w] = \begin{cases} dp[i-1][w], & \text{se } w_i > w \\ \max(dp[i-1][w], dp[i-1][w - w_i] + v_i), & \text{caso contrário} \end{cases}$$

A solução final é $dp[n][W]$.